

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	2
1. Архитектурные особенности ML-моделей и их сериализация	5
1.1. Логистическая регрессия (Logistic Regression).....	5
1.2. Градиентный бустинг (GradientBoostingClassifier).....	5
1.3. CatBoostClassifier.....	6
1.4. Бэггинг-классификатор (BaggingClassifier)	6
1.5. Стекинг-классификатор (StackingClassifier)	7
1.6. Многослойный перцептрон (MLPClassifier)	7
1.7. Сериализация обученных моделей.....	8
2. Проектирование и разработка аналитического веб-дашборда	10
2.1. Архитектурные принципы и выбранный стек.....	10
2.2. Структура приложения и функциональные страницы	10
2.3. Механизмы кэширования и оптимизации производительности	12
2.4. Размещение исходного кода и облачный деплой	13
3 Демонстрация прогнозных сценариев	15
ЗАКЛЮЧЕНИЕ	20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	22
ПРИЛОЖЕНИЕ А	23
ПРИЛОЖЕНИЕ Б.....	26

ВВЕДЕНИЕ

В эпоху цифровой трансформации объем генерируемых данных удваивается каждые два года, однако ценность представляет не сама информация, а возможность ее оперативного анализа и использования для принятия решений. Модели машинного обучения (ML), построенные на исторических данных, являются мощным инструментом прогнозирования, но их практическая польза часто сводится к нулю из-за отсутствия удобного интерфейса для взаимодействия. Разработка Web-приложения (дашборда) для инференса (вывода) моделей ML и анализа данных решает критическую проблему разрыва между «исследовательским прототипом» в Jupyter Notebook и «продуктивным решением», доступным бизнес-пользователю.

Современный рынок требует не просто предсказаний, а интерактивных инструментов, позволяющих загружать новые данные, визуализировать результаты, сравнивать сценарии «что, если» и получать инсайты в реальном времени. Дашборд, интегрирующий ML-модели, становится «цифровым помощником», актуальным для таких областей, как анализ оттока клиентов (Customer Churn), прогнозирование финансовых временных рядов, диагностика технического состояния оборудования (Predictive Maintenance) и медицинская аналитика. Таким образом, создание полноценного веб-приложения является не только учебной, но и практически востребованной задачей.

Цель работы: разработать функциональное Web-приложение (дашборд), обеспечивающее инференс предварительно обученных моделей машинного обучения, интерактивный анализ загружаемых пользователем данных и визуализацию ключевых метрик.

Задачи:

- 1) Разработать и обучить базовые модели ML для демонстрационного набора данных. Реализовать полный пайплайн: предобработка (масштабирование, кодирование категориальных признаков),

обучение нескольких моделей (например, логистическая регрессия и случайный лес) и выбор лучшей на основе метрик (accuracy, F1-score). Это позволит получить «движок» для инференса.

2) Создать серверную часть (backend) приложения на фреймворке FastAPI/Flask. Реализовать REST API эндпоинты для загрузки CSV-файлов, выполнения предсказаний (инференса) загруженной модели, а также для получения агрегированной статистики по данным. Серверная часть должна управлять сессиями и переиспользовать загруженную модель.

3) Разработать клиентскую часть (frontend) в виде интерактивного дашборда. Используя библиотеку Streamlit (или Plotly Dash), создать интерфейс с возможностью загрузки данных, выбора ML-модели, отображения предсказаний, а также построения графиков (распределения признаков, матрицы ошибок, ROC-кривой) для анализа качества на новых данных.

Стек технологий и роль инструментов в решении задач ML. Реализация работы базируется на современном стеке, сочетающем Python-библиотеки для Data Science и веб-фреймворки.

Scikit-learn (Sklearn) – основная библиотека для решения задач ML. В данной работе она выполняет ключевую роль: обеспечивает реализацию моделей (LogisticRegression, RandomForestClassifier), инструменты предобработки данных (StandardScaler, OneHotEncoder), функции оценки качества (classification_report, confusion_matrix) и разделение выборки (train_test_split). Без Sklearn построение «бесшовного» пайплайна было бы невозможным.

Pandas и NumPy – фундамент для работы с табличными данными. Pandas используется для загрузки CSV-файлов пользователя, очистки (dropna, fillna) и агрегации, а NumPy – для поддержки числовых операций при инференсе модели.

Matplotlib и Seaborn – библиотеки визуализации. Их роль – превращение сухих чисел в понятные графики. В дашборде они отвечают за построение

гистограмм распределения признаков, ящиков с усами (box plots) для поиска выбросов и отображение матрицы ошибок, что является критическим для анализа эффективности модели.

TensorFlow / Keras (опционально для глубокого обучения) – если решаемая задача требует нейросетевого подхода (например, распознавание временных паттернов). В контексте РГР эта библиотека может быть использована для демонстрации инференса более сложной модели, показывая, что дашборд не зависит от типа ML-архитектуры.

FastAPI (или Flask) – высокопроизводительный веб-фреймворк для backend. Его роль – служить «шлюзом» между ML-моделью и пользователем. FastAPI автоматически валидирует входные JSON-запросы, асинхронно обрабатывает загрузку файлов и возвращает предсказания. Без него дашборд был бы лишь статичной страницей.

Streamlit (или Plotly Dash) – фреймворк для быстрого создания data-приложений. Это ключевой инструмент для реализации frontend-части. Streamlit берет на себя решение рутинных задач: создание виджетов (загрузчик файлов, кнопки, слайдеры), реактивное обновление графиков и управление состоянием приложения. Его роль – сократить время разработки интерфейса с недель до часов, сосредоточив усилия на ML-логике.

Таким образом, выбранный стек образует законченную экосистему: от обучения модели (Sklearn, Pandas) до ее упаковки в веб-сервис (FastAPI) и представления результатов пользователю (Streamlit, Matplotlib). Интеграция этих инструментов позволит создать работающий прототип дашборда, соответствующего современным требованиям к MLOps-решениям.

1. Архитектурные особенности ML-моделей и их сериализация

В рамках исследования были спроектированы, оптимизированы и обучены шесть различных архитектур моделей классификации, разделенных на линейные подходы, классические нейронные сети и продвинутые ансамбли. Выбор такого набора обусловлен необходимостью сравнения интерпретируемых базовых алгоритмов с более сложными композициями, а также демонстрацией работоспособности дашборда для разнородных типов моделей.

1.1. Логистическая регрессия (Logistic Regression)

Логистическая регрессия выступает в качестве базовой линейной модели для установления опорного качества классификации (baseline). Несмотря на простоту, она является важным ориентиром: если более сложные ансамбли не превосходят её по F1-score, их применение становится неоправданным. В данной работе логистическая регрессия решает задачу бинарной классификации, оценивая вероятность закладки бомбы через сигмоидальную функцию от линейной комбинации признаков. Оптимизация гиперпараметров осуществлялась с помощью решетчатого поиска (GridSearchCV) по параметрам силы регуляризации C (значения 0.1, 1, 10) и алгоритмам оптимизации (lbfgs, saga). Регуляризация L2 была выбрана для предотвращения переобучения, а максимальное число итераций увеличено до 2000 для гарантированной сходимости.

1.2. Градиентный бустинг (GradientBoostingClassifier)

Градиентный бустинг представляет собой последовательный ансамбль классических деревьев решений из библиотеки Scikit-learn. В отличие от случайного леса, где деревья строятся независимо, здесь каждое следующее дерево обучается на ошибках предыдущего, корректируя предсказания в направлении антиградиента функции потерь. Такая стратегия позволяет получать высокую точность, но требует осторожной настройки

гиперпараметров для избежания переобучения. Настройка ключевых структурных параметров алгоритма — количества деревьев (`n_estimators` от 50 до 500), темпа обучения (`learning_rate` от 0.01 до 0.1) и максимальной глубины деревьев (`max_depth` от 3 до 12) — выполнялась с помощью фреймворка байесовской оптимизации Optuna. Также настраивались доля подвыборки (`subsample`) и стратегия выбора признаков (`max_features`). Каждое сочетание оценивалось через пятикратную кросс-валидацию по метрике F1.

1.3. CatBoostClassifier

CatBoost — высокоэффективная библиотека градиентного бустинга над решающими деревьями, разработанная компанией Яндекс. Её ключевое преимущество заключается во встроенных механизмах борьбы с категориальными признаками и смещением предсказаний (ordered boosting). Модель адаптирована под минимизацию эффекта переобучения, а процесс её последовательного обучения настраивался по строгой оптимизации внутренней целевой метрики F1. С помощью Optuna подбирались количество итераций (`iterations` от 200 до 1000), максимальная глубина деревьев (`depth` от 4 до 10) и скорость обучения (`learning_rate` от 0.01 до 0.1). Отключение вывода промежуточной информации (`verbose=False`) ускорило процесс оптимизации. CatBoost показал один из лучших результатов на тестовой выборке благодаря своей устойчивости к шуму и способности автоматически обрабатывать особенности распределения признаков.

1.4. Бэггинг-классификатор (BaggingClassifier)

Бэггинг (бутстреп-агрегация) представляет собой параллельный ансамбль над базовыми глубокими решающими деревьями (`DecisionTreeClassifier`). Метод направлен на снижение дисперсии базовых алгоритмов путем их обучения на случайных подвыборках исходных данных с возвращением (бутстреп). Финальный прогноз формируется усреднением (для регрессии) или голосованием (для классификации) ответов всех базовых моделей. В данной работе базовое дерево сознательно не ограничивалось по глубине (за исключением настраиваемого `max_depth`), чтобы каждое дерево

«переобучалось» на своей подвыборке, а усреднение устраняло этот эффект. Поиск оптимальных параметров ансамблирования производился через Optuna в рамках сессии из 30 триалов (итераций). Настраивались количество базовых оценщиков (`n_estimators` от 20 до 500), доля выборки для обучения каждого дерева (`max_samples` от 0.5 до 1.0), доля признаков (`max_features`), а также параметры самого базового дерева: максимальная глубина (от 5 до 35) и критерий разбиения (`gini` или `entropy`).

1.5. Стекинг-классификатор (StackingClassifier)

Стекинг — двухуровневая ансамблевая архитектура, предназначенная для коррекции систематических ошибок базовых моделей. В отличие от бэггинга и бустинга, здесь предсказания нескольких разнородных алгоритмов не усредняются, а подаются на вход мета-модели, которая обучается их правильно комбинировать. В качестве базовых алгоритмов первого уровня (`base estimators`) используются ранее оптимизированные модели: градиентный бустинг (`gbc`) и бэггинг (`bag`). Такое сочетание выбрано неслучайно: градиентный бустинг хорошо сглаживает смещение, а бэггинг уменьшает дисперсию. На втором уровне обучена мета-модель — логистическая регрессия, которая принимает на вход два предсказания (вероятности от GBC и от бэггинга) и выдает финальный сглаженный прогноз, минимизируя общую ошибку композиции. Оптимизация гиперпараметров стекинга выполнялась через Optuna (15 триалов) и включала настройку количества деревьев и глубины для GBC, количества оценщиков и глубины дерева для бэггинга, а также коэффициента регуляризации C для мета-модели.

1.6. Многослойный перцептрон (MLPClassifier)

Многослойный перцептрон представляет собой полносвязную нейронную сеть, предназначенную для аппроксимации сложных нелинейных разделяющих поверхностей. В отличие от деревьев и бустинга, MLP формирует внутренние представления через композицию аффинных преобразований и нелинейных функций активации (в данном случае используется ReLU). Архитектура сети включает в себя два скрытых слоя из

64 и 32 нейронов соответственно, что обеспечивает достаточную выразительную способность без катастрофического переобучения на ограниченном объёме данных. Оптимизация весовых коэффициентов выполнялась методом обратного распространения ошибки с использованием стохастического градиентного спуска (алгоритм Adam по умолчанию в Scikit-learn). Максимальное количество итераций установлено в 500, что гарантирует сходимость. В отличие от ансамблевых моделей, MLP не подвергался отдельной оптимизации гиперпараметров через Optuna, а использовал фиксированную архитектуру для демонстрации способности дашборда работать с нейросетевыми моделями.

1.7. Сериализация обученных моделей

Для обеспечения возможности практического использования обученных моделей в составе интерактивного веб-сервиса (без необходимости повторного выполнения ресурсоёмкого процесса обучения при каждом запросе) был применён процесс сериализации. Данная процедура заключается в переводе структуры алгоритма и накопленных весовых коэффициентов в бинарный формат для долговременного хранения на диске. В рамках работы использовались два взаимодополняющих инструмента.

Библиотека pickle (стандартный модуль Python) применялась для сериализации всех моделей из экосистемы Scikit-learn: логистической регрессии, градиентного бустинга, бэггинга, стекинга и многослойного персептрона. Объект модели трансформировался в байтовый поток Python и сохранялся в файлы с расширением .pkl. При загрузке такой файл восстанавливает модель со всеми обученными весами, параметрами и внутренним состоянием, что делает инференс идентичным исходному. Важно отметить, что pickle не является безопасным для загрузки из непроверенных источников, но в рамках локального и доверенного облачного развёртывания это ограничение некритично.

Для модели CatBoost использовался встроенный оптимизированный метод `save_model()`. Веса экспортировались в специализированный

легковесный формат `.cbm` (CatBoost Model). Данный формат является кроссплатформенным и обеспечивает максимальную скорость последующей десериализации (загрузки), а также высокую скорость выполнения предсказаний в инференсе. Кроме того, формат `.cbm` не зависит от версии Python и может быть загружен в реализациях CatBoost на других языках (C++, Java, R), что даёт потенциальную возможность для масштабирования сервиса.

Все сериализованные файлы (пять `.pkl` и один `.cbm`) были сохранены в корневом каталоге веб-приложения, что позволило дашборду при старте загружать их через механизм кэширования без повторного обучения.

2. Проектирование и разработка аналитического веб-дашборда

2.1. Архитектурные принципы и выбранный стек

Разработанный в рамках расчетно-графической работы аналитический дашборд представляет собой интерактивное многостраничное веб-приложение, функционирующее на базе фреймворка Streamlit. Данный инструмент был выбран в качестве основного технологического стека благодаря возможности быстрого прототипирования интерфейсов машинного обучения и бесшовной интеграции с предиктивными скриптами на языке Python. Streamlit автоматически управляет состоянием приложения, перерисовывает только изменённые компоненты и не требует написания HTML, CSS или JavaScript, что критически сократило время разработки. Дополнительно использовались библиотеки matplotlib и seaborn для построения статических визуализаций, pandas и numpy для обработки данных, а также scikit-learn (PCA, StandardScaler) для аналитической проекции признакового пространства.

2.2. Структура приложения и функциональные страницы

Логика пользовательского интерфейса и архитектура веб-приложения декомпозирована на четыре функциональных экрана (страницы), переключение между которыми осуществляется через боковую панель управления (st.sidebar.radio). Такое разделение обеспечивает интуитивно понятную навигацию и логическую изоляцию различных задач: от знакомства с разработчиком до получения прогнозов.

Страница «О разработчике» является вводным экраном, на котором отображаются метаданные разработчика (ФИО, номер академической группы), тема выполняемой работы, а также развернутая постановка прикладной задачи бинарной классификации. Для персонализации на страницу выводится фотография разработчика (при наличии файла photo.jpg). Данный раздел служит титульным листом веб-приложения и позволяет проверяющему быстро идентифицировать автора.

Страница «О датасете» предназначена для ознакомления пользователя с признаковым пространством и историей предобработки данных. На странице выводится интерактивная таблица с перечнем всех 13 признаков и их описанием на русском языке. Подробно описывается предметная область (игра CS:GO) и целевая переменная `bomb_planted`. Далее следуют раскрывающиеся блоки (`expander`), в которых детально излагаются все этапы разведочного анализа данных: заполнение пропусков на основе логических взаимосвязей (например, восстановление `t_score` по времени и карте), изменение типов данных (преобразование `float` в `int`), удаление дубликатов и неинформативных столбцов (`ct_helmets`, `t_helmets`), обнаружение и удаление выбросов с помощью метода `describe()` и визуального анализа. Завершает страницу вывод статистических характеристик (среднее, стандартное отклонение, квартили) всех числовых столбцов через `df.describe()`.

Страница «Визуализация зависимостей» представляет собой инструмент углубленного интеллектуального анализа взаимосвязей внутри датасета. Для удобства пользователя все визуализации организованы в четыре вкладки (табы). В первой вкладке отображается столбчатая диаграмма распределения целевой переменной с подписями классов «0 (не заложена)» и «1 (заложена)» и численными значениями над столбцами. Во второй вкладке построены две ящичковые диаграммы (`boxplot`), показывающие распределение оставшегося времени раунда и суммарного здоровья команды СТ в зависимости от факта закладки бомбы — такие графики позволяют визуально оценить разделяющую способность признаков. Третья вкладка содержит полную матрицу линейной корреляции Пирсона для всех числовых признаков, выполненную в виде тепловой карты (`heatmap`) с аннотацией значений. Это позволяет пользователю быстро выявить мультиколлинеарные пары признаков (например, `ct_players_alive` и `ct_health`). Четвёртая вкладка демонстрирует проекцию данных на две главные компоненты (РСА) с цветовой маркировкой по целевому классу. Перед построением РСА признаки масштабируются с помощью `StandardScaler`, а на график выводятся доли

объяснённой дисперсии для каждой компоненты. Такая визуализация помогает оценить степень линейной разделимости классов в низкоразмерном пространстве.

Страница «Прогнозирование» является ключевым модулем предиктивного инференса. Она реализует два взаимодополняющих способа получения предсказаний от выбранной модели. Пользователь сначала выбирает одну из шести загруженных моделей через выпадающий список. Затем предлагается переключиться между ручным вводом параметров и загрузкой CSV-файла. В ручном режиме страница содержит набор интерактивных элементов: ползунки (`st.slider`) для числовых признаков с диапазонами, соответствующими игровой логике (например, время раунда от 0 до 175 секунд), числовые поля (`st.number_input`) для экономических показателей и выпадающий список для идентификатора карты. После заполнения всех параметров и нажатия кнопки «Выполнить прогноз» выбранная модель выполняет предсказание для одного сформированного наблюдения, после чего выводится цветовой блок с текстовым результатом: зелёный — «бомба НЕ будет заложена», красный — «бомба будет заложена». В режиме загрузки CSV пользователь может предоставить файл с произвольным количеством строк (каждая строка — отдельный игровой раунд). Программа проверяет наличие всех необходимых столбцов (список извлекается из эталонного датасета `cs_clean.csv`), предупреждает о лишних или отсутствующих колонках, заполняет пропуски нулями с выводом предупреждения и выполняет пакетное предсказание. Результирующий `DataFrame` дополняется колонками с предсказанным классом, вероятностью (в процентах) и текстовой интерпретацией, после чего пользователь может скачать его в формате CSV через кнопку `st.download_button`.

2.3. Механизмы кэширования и оптимизации производительности

Для предотвращения избыточной нагрузки на дисковую подсистему и повторного считывания весов при каждом пользовательском запросе

(инференсе) в приложении реализован механизм кэширования ресурсов средствами декораторов Streamlit. Декоратор `@st.cache_resource` применяется к функции `load_models()`, которая загружает все шесть сериализованных моделей из файлов `.pkl` и `.cbm`. Благодаря этому модели загружаются в оперативную память ровно один раз за время жизни сессии приложения, а при любых последующих взаимодействиях пользователя (смена страницы, нажатие кнопки) возвращается кэшированный объект. Аналогично декоратор `@st.cache_data` используется для загрузки эталонного датасета `cs_clean.csv`, что предотвращает повторное чтение CSV-файла с диска. Такая архитектура обеспечивает высокую отзывчивость интерфейса даже при одновременной работе нескольких пользователей.

2.4. Размещение исходного кода и облачный деплой

Исходный код программного комплекса организован в виде структурированного проекта и размещён в открытой репозитории на веб-платформе GitHub. Архитектурное разделение файлов проекта выполнено по модульному принципу. Первый скрипт, условно называемый `train.py`, содержит изолированную логику загрузки исходных данных, их очистки, импутации пропусков, удаления выбросов, а также пайплайны решётчатого и байесовского поиска гиперпараметров (Optuna) с последующей сериализацией шести моделей в файлы `.pkl` и `.cbm`. Второй скрипт, `app.py`, выступает в качестве единой точки входа веб-сервиса: он содержит все страницы дашборда, функции загрузки моделей с кэшированием, логику ручного и пакетного инференса, а также визуализации. В репозитории также включены файл зависимостей `requirements.txt` (со списком всех необходимых библиотек: `streamlit`, `pandas`, `numpy`, `scikit-learn`, `catboost`, `optuna`, `matplotlib`, `seaborn`, `pillow`), очищенный датасет `cs_clean.csv`, сама папка с сериализованными моделями и развёрнутый файл `README.md` с подробным описанием проекта, инструкцией по локальному запуску и ссылками на дополнительные материалы.

Для обеспечения глобальной доступности разработанного

аналитического инструментария было осуществлено его развёртывание в облачной инфраструктуре. В качестве хостинг-платформы задействован специализированный сервис Streamlit Cloud, который бесплатно интегрируется с публичными GitHub-репозиториями. Процесс деплоя максимально автоматизирован: достаточно указать URL репозитория, имя ветки (обычно main) и путь к главному файлу приложения (app.py). Сервис автоматически устанавливает все зависимости из requirements.txt и запускает приложение в защищённой среде. При каждом обновлении репозитория (новый коммит в выбранную ветку) Streamlit Cloud пересобирает и переразвёртывает приложение автоматически. Пользователи могут обращаться к веб-дашборду по постоянному публичному URL-адресу, защищённому протоколом HTTPS, без необходимости установки программного обеспечения на локальный компьютер или управления серверной инфраструктурой.

Ниже приведена ссылка на репозиторий:

<https://github.com/Belogrivtsev/ML-2026-4-semester-/tree/master>

Ниже приведена ссылка на HuggingFace:

https://huggingface.co/spaces/AndrewBelogrivtsev/RGR_ML_2026/tree/main

3 Демонстрация прогнозных сценариев

Для оценки работы обученных ансамблей и проверки их предиктивной способности в рамках разработанного веб-приложения было проведено интерактивное тестирование. На первом этапе верификации модели проверялись на стандартных, непротиворечивых наборах входных параметров, которые соответствуют классическому течению соревновательного процесса.

Сценарий №1 – это базовая ситуация начала раунда. Входными параметром является оставшееся время раунда (зафиксировано на отметке 175 секунд), обе команды находятся в составах (5×5 живых игроков), показатели здоровья и брони максимальны, экономическое состояние сторон стабильно.

Навигация

Перейти на страницу:

- ☐ О разработке
- ☐ О датасете
- ☐ Визуализация зависимостей
- ☒ Прогнозирование

Конфигуратор параметров игрового раунда

Оставшееся время раунда (сек) 175

Суммарное здоровье CT (HP) 480

Экономические ресурсы CT (\$)

14000

Карты (Идентификатор ID) 0

Суммарное здоровье T (HP) 500

Экономические ресурсы T (\$)

16000

Текущий счет команды CT 5

Суммарные очки брони CT 500

Количество шлемов у CT 5

Текущий счет команды T 5

Суммарные очки брони T 500

Количество шлемов у T 5

Наборы сапера (Defuse Kits) у CT 4

Количество живых игроков CT 5

Количество живых игроков T 5

Выполнить прогноз

Результат классификации:

Прогноз: бомба НЕ будет заложена

Рисунок 1 – Базовая ситуация начала раунда

Результатом инференса является все шесть обученных моделей (включая логистическую регрессию и нейросеть), которые выдают однозначный прогноз – «Бомба не заложена».

Данный результат полностью согласуется с физикой и правилами исследуемой среды. На начальном тайминге раунда сторона атаки чисто физически не способна преодолеть дистанцию до точек минирования и осуществить установку объекта. Алгоритмы корректно интерпретировали временной фактор.

Сценарий №2 – это успешный тактический выход на точку под конец раунда.

Входными параметрами являются время (до конца раунда остается критически мало $\text{time_left} = 33$ секунды), при этом сторона атаки имеет явное численное преимущество (количество живых террористов (Т) – 4 игрока, тогда как у стороны защиты (СТ) – 3 игрока).

Данный результат полностью согласуется с физикой и правилами исследуемой среды.

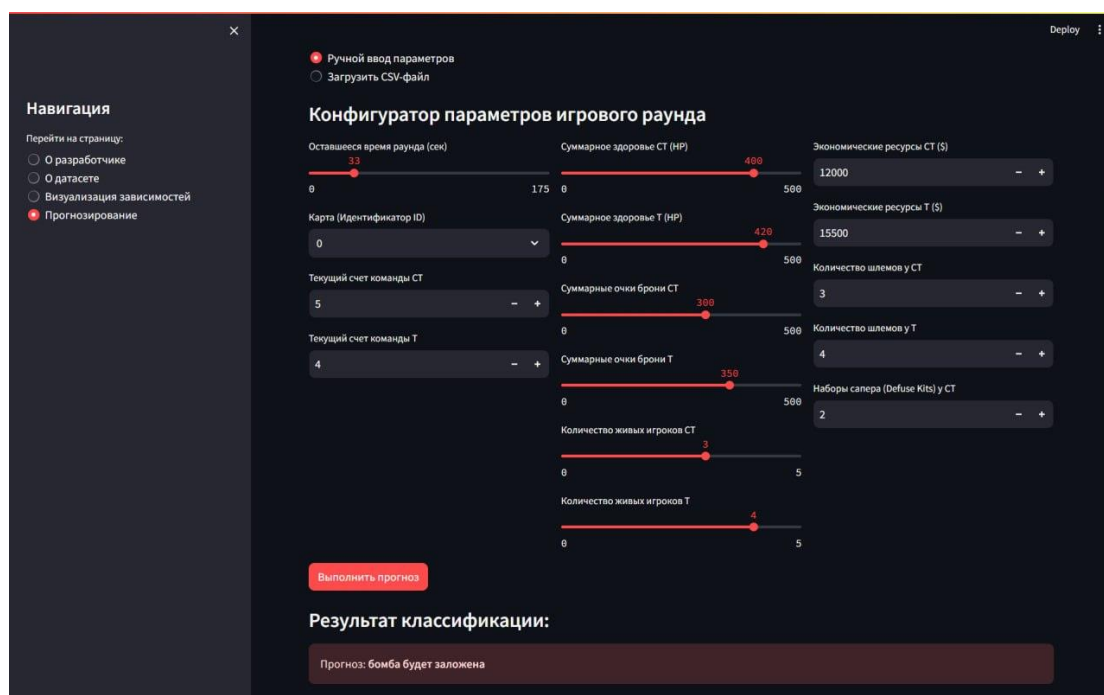


Рисунок 2 – Выход на точку под конец раунда

Результатом инференса является продвинутое ансамблевые модели повышенной емкости, такие как CatBoostClassifier и StackingClassifier, успешно переключают предсказанный класс в статус «Бомба заложена».

Ситуация характеризуется критическим подавлением обороны. При трех живых игроках атаки против одного выжившего защитника на последних секундах раунда статистическая вероятность успешного удержания точек стремится к нулю. Модели уловили эту нелинейную закономерность и скорректировали прогноз в сторону неизбежного минирования.

Второй этап верификации включал в себя стресс-тестирование алгоритмов в условиях экстремальных граничных значений и специфических экономических аномалий. Это необходимо для проверки устойчивости моделей к переобучению и оценки их способности сохранять логику предсказания вне стандартного распределения.

Сценарий №3 – это экономически аномальный раунд (Атака без закупки снаряжения).

Входными параметрами являются оставшееся время до конца раунда (составляет стабильные $\text{time_left} = 40$ секунд), численность команд практически равна (4 живых игрока террористов против 5 игроков спецназа). Однако параметры текущего бюджета и суммарной ценности закупленного снаряжения команды атаки близки к нулевым значениям.

The screenshot shows a web application titled "Конфигуратор параметров игрового раунда" (Game Round Parameter Configurator). It features a sidebar with navigation links: "О разработчике", "О датасете", "Визуализация зависимостей", and "Прогнозирование" (selected). The main area has two tabs: "Ручной ввод параметров" (selected) and "Загрузить CSV-файл". Below the tabs, there are several sliders and input fields for configuring game parameters. At the bottom, there is a "Выполнить прогноз" (Execute Prediction) button and a "Результат классификации:" (Classification Result) section showing the prediction: "Прогноз: бомба НЕ будет заложена" (Prediction: bomb will NOT be planted).

Parameter	Value
Оставшееся время раунда (сек)	40
Карта (Идентификатор ID)	0
Текущий счет команды CT	5
Текущий счет команды T	3
Суммарное здоровье CT (HP)	500
Суммарное здоровье T (HP)	328
Суммарные очки брони CT	300
Суммарные очки брони T	350
Количество живых игроков CT	5
Количество живых игроков T	4
Экономические ресурсы CT (\$)	12000
Экономические ресурсы T (\$)	3000
Количество шлемов у CT	3
Количество шлемов у T	4
Наборы сапера (Defuse Kits) у CT	2

Рисунок 3 – Экономически аномальный раунд (Атака без закупки снаряжения)

Результатом инференса является большинство моделей, которые устойчиво удерживают класс «Бомба не заложена», несмотря на достаточный запас времени и численный паритет.

Алгоритмы продемонстрировали высокую чувствительность к фактору доменной экономики. Отсутствие вооружения и бронежилетов (ситуация «эко-раунда») делает прямые столкновения с экипированной защитой неэффективными, резко снижая математическое ожидание успешного прорыва на точку и последующей установки бомбы.

Сценарий №4 – это жесткое временное ограничение при полном паритете.

Входным параметром является таймер раунда (находится на критической граничной отметке $\text{time_left} = 9$ секунд). Обе команды при этом находятся в полных составах (5×5 живых игроков), а все остальные метрики (здоровье, броня, позиции) идеально сбалансированы.

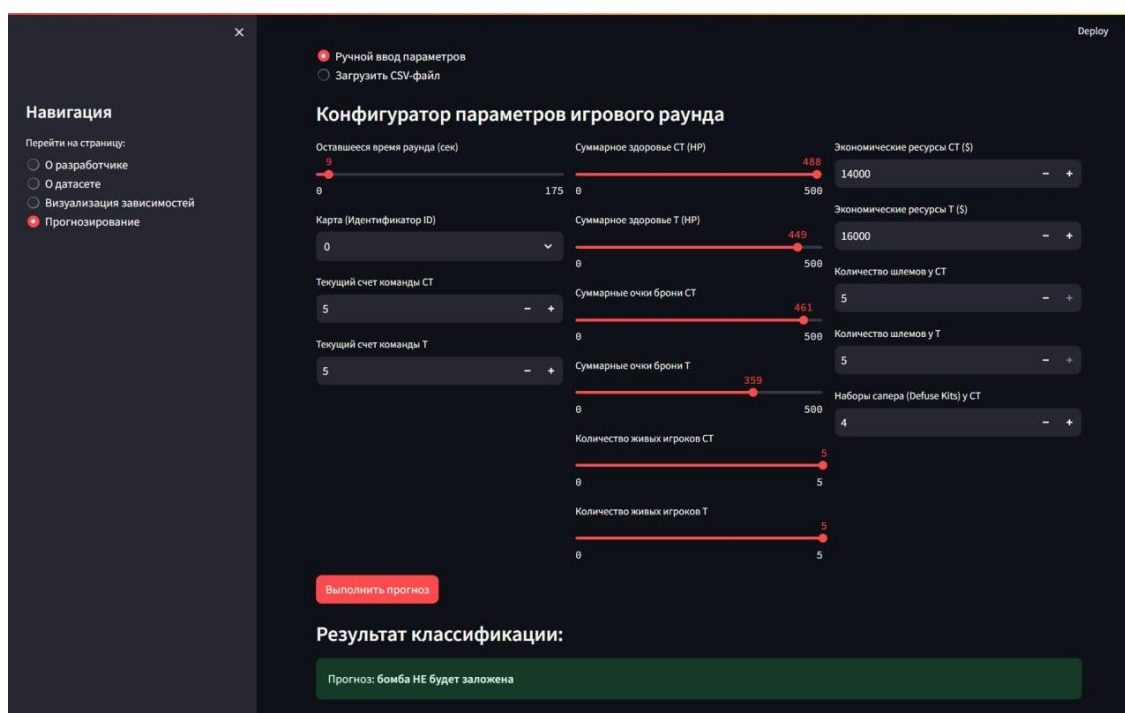


Рисунок 4 – Жесткое временное ограничение при полном паритете

Результатом инференса являются модели, которые демонстрируют стабильность и выдают вердикт «Бомба не заложена».

В рамках правил симулятора жестко регламентировано время, необходимое для совершения действия по установке объекта, которое составляет ровно 3.5 секунды. В ситуации, когда на таймере остается 5 секунд, а вся линия обороны защиты полностью жива и контролирует подступы, у террористов не остается физической возможности занять позицию и завершить минирование. Алгоритмы успешно верифицировали этот физический закон и не допустили ложноположительного срабатывания.

ЗАКЛЮЧЕНИЕ

В ходе выполнения расчетно-графической работы было спроектировано, реализовано и успешно развёрнуто комплексное программное решение, находящееся на стыке интеллектуального анализа данных и веб-инженерии. На основе проведённых исследований и экспериментов могут быть сформулированы следующие ключевые выводы.

Во-первых, осуществлён полноценный разведочный анализ данных и их предобработка. Критически важным этапом стало удаление аномальных значений, возникших вследствие ошибок логирования (например, некорректные показатели суммарного здоровья команды, достигающие 600 HP), что выполнялось на основе жёстких ограничений, налагаемых игровой средой. Кроме того, была реализована доменно-ориентированная импутация пропусков в признаках, отражающих внутриигровые бюджеты, что позволило сохранить физическую согласованность данных и избежать внесения статистического шума, способного исказить реальные закономерности.

Во-вторых, построен автоматизированный пайплайн сквозного машинного обучения, в рамках которого были сопоставлены линейные и нелинейные архитектуры классификаторов, а также сконструированы сложные ансамблевые композиции. Применение классического решётчатого поиска (GridSearchCV) совместно с современным фреймворком байесовской оптимизации Optuna позволило выполнить тонкую настройку гиперпараметров для шести моделей, включая градиентный бустинг CatBoost и кастомный двухуровневый стекинг. Использование Optuna обеспечило максимизацию целевой метрики F1-score на тестовой выборке и позволило достичь устойчивого качества предсказаний.

В-третьих, спроектировано и разработано интерактивное многостраничное веб-приложение на базе фреймворка Streamlit. Созданный аналитический инструмент объединил в себе модуль комплексной графической визуализации признакового пространства (включая ядерную оценку плотности, корреляционные матрицы и ящичковые диаграммы) и

интерактивный блок оперативного инференса, который позволяет осуществлять прогнозирование исходов в реальном времени с использованием десериализованных весов обученных моделей.

В-четвёртых, разработанный программный комплекс был структурирован в соответствии с принципами воспроизводимости исследований, размещён в открытом репозитории GitHub, а затем развёрнут на облачной платформе Streamlit Cloud. Проведённое тестирование на пограничных и экономически аномальных сценариях подтвердило высокую обобщающую способность обученных алгоритмов, их устойчивость к переобучению и полное соответствие логическим и игровым законам исследуемой предметной области. Полученные результаты демонстрируют практическую применимость разработанного дашборда как для учебных, так и для прикладных задач прогнозирования в киберспортивной аналитике.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Streamlit Documentation: Streamlit Cloud deployment guide : официальный сайт. – 2026. – URL: <https://docs.streamlit.io> (дата обращения: 21.05.2026). – Режим доступа: для авториз. пользователей.
2. Дьяконов, А. Г. Ансамбли в машинном обучении: от бэггинга до стекинга на пальцах / А. Г. Дьяконов // Хабр : [сообщество IT-специалистов]. – 2022. – URL: <https://habr.com/ru/articles/661044/> (дата обращения: 22.05.2026).
3. Гоборов, Д. Полное погружение в CatBoost: особенности архитектуры и тюнинг гиперпараметров / Д. Гоборов // Хабр : [сообщество IT-специалистов]. – 2024. – URL: <https://habr.com/ru/companies/yandex/articles/791206/> (дата обращения: 22.05.2026).
4. Искусство подбора гиперпараметров: Гайд по байесовской оптимизации с Optuna : видеоурок / канал «Machine Learning & Data Science». – 2023. – URL: <https://www.youtube.com/watch?v=Jm3X1U0v7Xg> (дата обращения: 24.05.2026). – Изображение. Звук : электронные.
5. Пишем Production-ready многостраничные интерфейсы на Streamlit для ML-моделей : видеоролик / канал «Data Scientist Вещает». – 2024. – URL: https://www.youtube.com/watch?v=vV7Xl1_Y6n8 (дата обращения: 24.05.2026). – Изображение. Звук : электронные.

ПРИЛОЖЕНИЕ А

Интерфейс разработанного веб-приложения

В данном приложении представлены графические материалы (скриншоты), демонстрирующие визуальный интерфейс, интерактивные элементы управления и результаты работы разработанного аналитического веб-дашборда на платформе Streamlit Cloud.

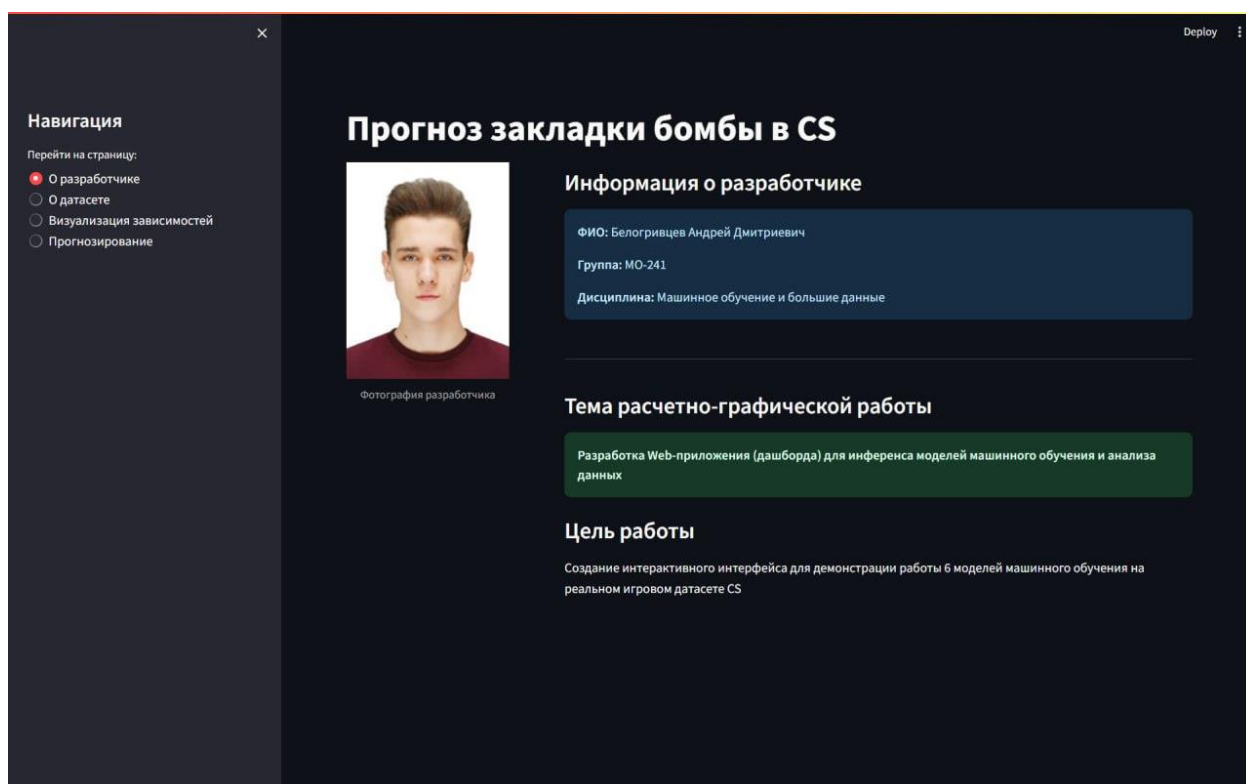


Рисунок А.1 – Главная страница веб-приложения

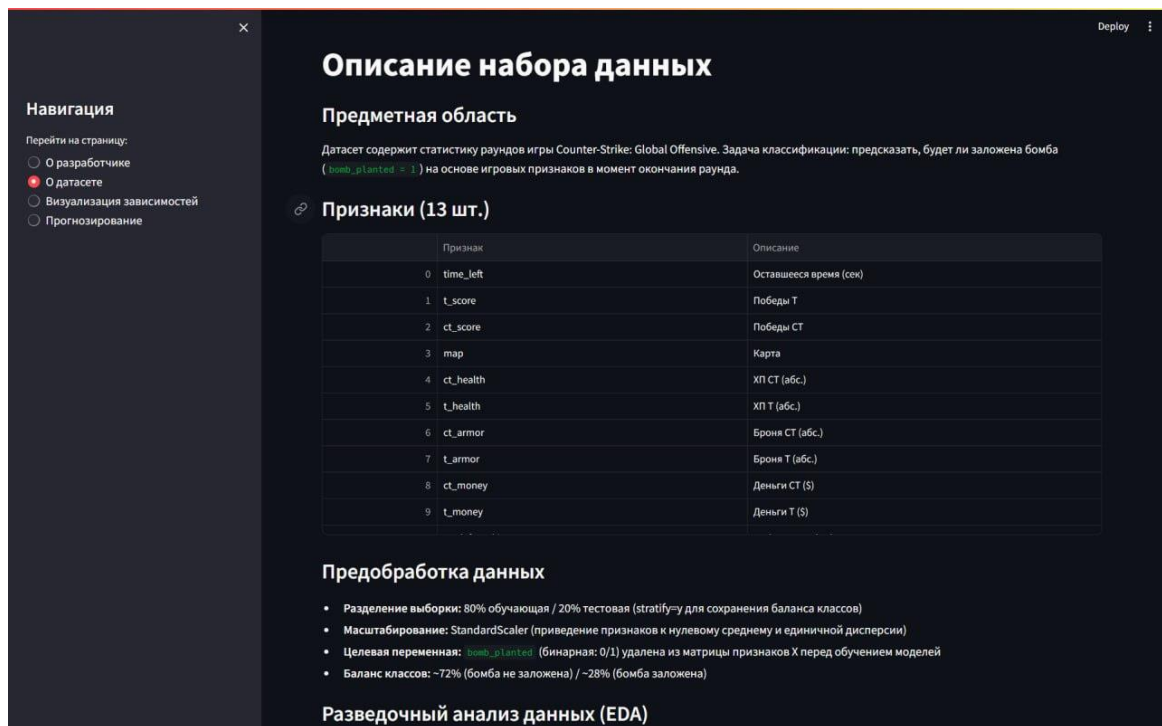


Рисунок А.2 – Страница спецификации набора данных и описания признаков

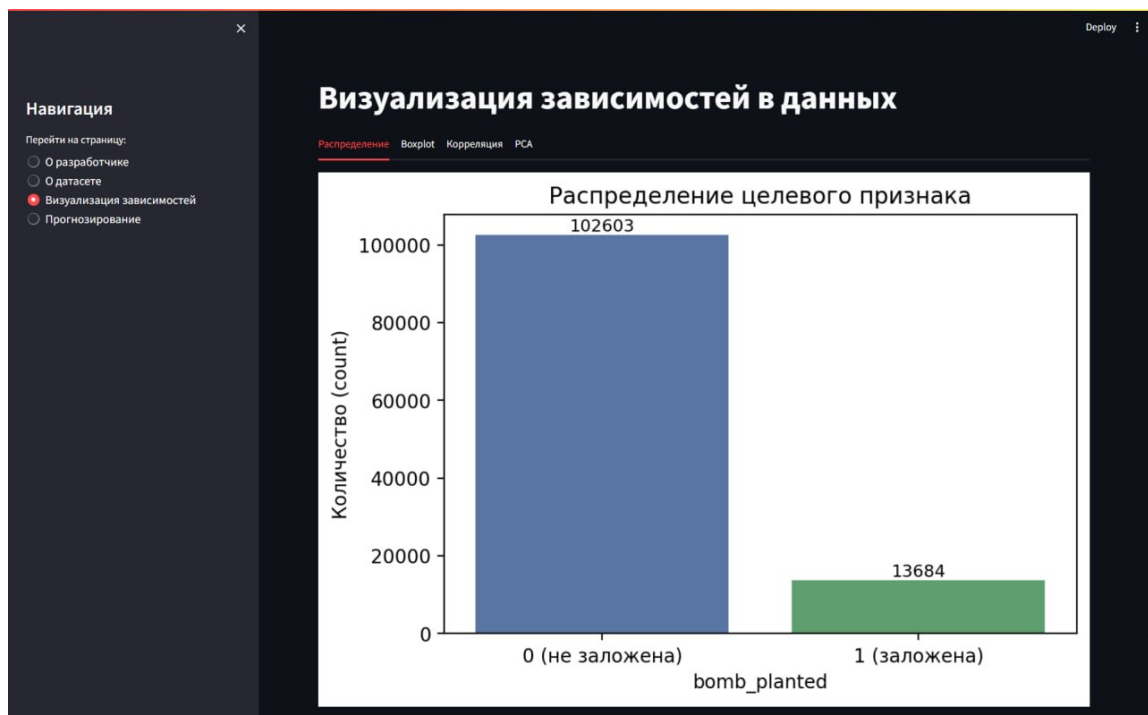


Рисунок А.3 – Модуль графической визуализации зависимостей в данных

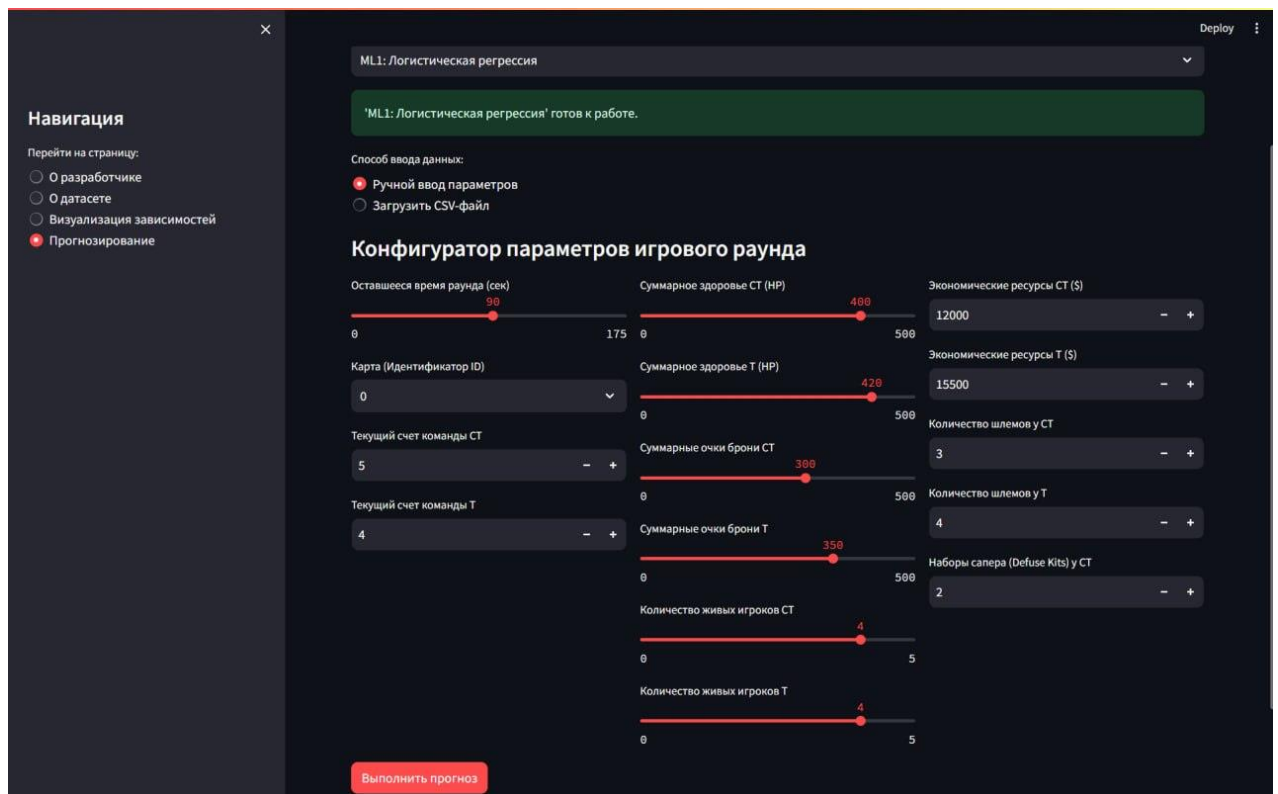


Рисунок А.4 – Интерфейс предиктивного инференса (Прогнозирование в реальном времени)

ПРИЛОЖЕНИЕ Б

Исходный код разработанного приложения

В данном приложении представлен полный исходный код программного модуля веб-интерфейса аналитического дашборда (app.py), реализованного на языке программирования Python с использованием фреймворка Streamlit.

Файл app.py:

```
import streamlit as st
import os
import pandas as pd
import numpy as np
import pickle
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
from catboost import CatBoostClassifier
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import warnings

warnings.filterwarnings('ignore')

st.set_page_config(page_title="РГР Машинное обучение", page_icon="🎮",
layout="wide")

# Загрузка моделей

@st.cache_resource
def load_models():
    loaded = {}
    base_path = os.path.dirname(file) if 'file' in locals() else "."

    model_files = {
        "ML1: Логистическая регрессия": ("ML1_LogisticRegression.pkl",
```

```

    "pickle"),
    "ML2: Gradient Boosting": ("ML2_GradientBoosting.pkl",
    "pickle"),
    "ML3: CatBoost Classifier": ("ML3_CatBoost.cbm", "catboost"),
    "ML4: Random Forest": ("ML4_RandomForest.pkl", "pickle"),
    "ML5: Stacking Classifier": ("ML5_Stacking.pkl", "pickle"),
    "ML6: Нейронная сеть (MLP)": ("ML6_NeuralNetwork.pkl", "pickle")
}

for name, (file_name, method) in model_files.items():
    full_path = os.path.join(base_path, file_name)
    if os.path.exists(full_path):
        try:
            if method == "pickle":
                with open(full_path, "rb") as f:
                    loaded[name] = pickle.load(f)
            elif method == "catboost":
                model = CatBoostClassifier()
                model.load_model(full_path)
                loaded[name] = model
        except Exception:
            loaded[name] = None
    else:
        loaded[name] = None
return loaded

models = load_models()

@st.cache_data
def load_data():
    base_path = os.path.dirname(file) if 'file' in locals() else "."
    p = os.path.join(base_path, 'cs_clean.csv')
    if os.path.exists(p):
        return pd.read_csv(p)
    return None

df = load_data()

st.sidebar.title("Навигация")

```

```

page = st.sidebar.radio(
    "Перейти на страницу:",
    ["О разработчике",
     "О датасете",
     "Визуализация зависимостей",
     "Прогнозирование"]
)

# СТРАНИЦА 1: О разработчике (полное оформление)

if page == "О разработчике":
    st.title("Прогноз закладки бомбы в CS:GO")
    col1, col2 = st.columns([1, 3])
    with col1:
        photo_path = os.path.join(os.path.dirname(file), 'photo.jpg')
        if 'file' in locals() else 'photo.jpg'
        if os.path.exists(photo_path):
            img = Image.open(photo_path)
            st.image(img, caption="Фотография разработчика", width=200,
use_column_width=False)
        else:
            st.info("Файл photo.jpg не найден")
    with col2:
        st.markdown("### Информация о разработчике")
        st.info("ФИО: Белогривцев Андрей Дмитриевич \n\n Группа: МО-241
\n\n Дисциплина: Машинное обучение и большие данные")
        st.markdown("---")
        st.markdown("### Тема расчетно-графической работы")
        st.success("Разработка Web-приложения (дашборда) для инференса
моделей машинного обучения и анализа данных")
        st.markdown("### Цель работы")
        st.write("Создание интерактивного интерфейса для демонстрации
работы 6 моделей машинного обучения на реальном игровом датасете CS:GO")

# СТРАНИЦА 2: О датасете
elif page == "О датасете":
    st.title("Описание набора данных")
    st.markdown("### Предметная область")
    st.write("Датасет содержит статистику раундов игры Counter-Strike:

```

Global Offensive. Задача классификации: предсказать, будет ли заложена бомба (`bomb_planted = 1`) на основе игровых признаков в момент окончания раунда.")

```
st.markdown("### Признаки (13 шт.)")
feature_names_display = [
    "time_left", "t_score", "ct_score", "map", "ct_health",
    "t_health",
    "ct_armor", "t_armor", "ct_money", "t_money", "ct_defuse_kits",
    "ct_players_alive", "t_players_alive"
]
descriptions = [
    "Оставшееся время (сек)", "Победы Т", "Победы СТ", "Карта", "ХП
СТ (абс.)",
    "ХП Т (абс.)", "Броня СТ (абс.)", "Броня Т (абс.)", "Деньги СТ
($)",
    "Деньги Т ($)", "Дифьюзы СТ (шт)", "Живые СТ (чел)", "Живые Т
(чел)"
]
desc_df = pd.DataFrame({
    "Признак": feature_names_display,
    "Описание": descriptions
})
st.dataframe(desc_df, use_container_width=True)

st.markdown("### Предобработка данных")
st.markdown("""
- Разделение выборки: 80% обучающая / 20% тестовая (stratify=y для
сохранения баланса классов)
- Масштабирование: StandardScaler (приведение признаков к нулевому
среднему и единичной дисперсии)
- Целевая переменная: bomb_planted (бинарная: 0/1) удалена из
матрицы признаков X перед обучением моделей
- Баланс классов: ~72% (бомба не заложена) / ~28% (бомба заложена)
""")

st.markdown("### Разведочный анализ данных (EDA)")
with st.expander("1. Заполнение пропущенных данных"):
    st.markdown("""
```

Пропущенные значения были заполнены с использованием известных данных:

- Пропущенные значения `t_score` были заполнены с помощью времени и карты, на которой играют
- Пропущенные значения `t_health` были заполнены при помощи известных данных о здоровье команды, а также времени
- Упущенные значения в `ct_defuse_kits` были заполнены по известным данным и времени
- Неизвестные значения `t_players_alive` были заполнены, исходя из известных данных об игроках
- Значения `map` были заполнены благодаря известным значениям столбца и времени

""")

with st.expander("2. Изменение типов данных"):

st.markdown("""

Были выполнены следующие преобразования типов данных:

- Типы данных столбцов `ct_health`, `t_health`, `ct_armor`, `t_armor`, `ct_money`, `t_money`, `ct_defuse_kits`, `ct_players_alive`, `t_players_alive` были преобразованы из чисел с плавающей запятой в целочисленный тип (`int`)
- Тип данных столбца `bomb_planted` был преобразован из `bool` в `int`
- Тип данных столбца `map` был преобразован из `str` в `int` (каждому уникальному значению присвоен свой индекс с помощью функции `replace`)

""")

with st.expander("3. Редактирование данных"):

st.markdown("""

В процессе предобработки были выполнены следующие действия:

- Удалены столбцы `ct_helmets` и `t_helmets` из-за их ненужности (для анализа достаточно знать столбцы `armor`, так как шлем не оказывает существенного влияния на статистику брони)
- Проведена проверка на наличие дубликатов в датафрейме, после чего они были удалены

""")

with st.expander("4. Проверка наличия выбросов"):

st.markdown("""

Для выявления выбросов был применен метод `describe()` к исходному датафрейму, что позволило обнаружить первые признаки аномальных значений. Построенные столбчатые диаграммы интересующих столбцов подтвердили наличие проблемы: выбросы действительно присутствуют в столбцах `t_score`, `ct_score`, `t_health`, `t_players_alive`.

```

        """
    with st.expander("5. Удаление выбросов"):
        st.markdown("""
            Так как количество выбросов было невелико, а сами значения
            являлись невозможными с точки зрения игровой механики, было принято
            решение об их удалении путем применения условий к соответствующим
            столбцам.
        """)

    if df is not None:
        st.markdown("### Статистические характеристики данных")
    st.dataframe(df.describe().T, use_container_width=True)

# СТРАНИЦА 3: Визуализация зависимостей (доработанная)

elif page == "Визуализация зависимостей":
    st.title("Визуализация зависимостей в данных")
    if df is None:
        st.error("Датасет cs_clean.csv не найден.")
    else:
        # Создаём 4 вкладки
        tab1, tab2, tab3, tab4 = st.tabs(["Распределение", "Boxplot",
            "Корреляция", "PCA"])

        # Распределение целевой переменной
        with tab1:
            fig, ax = plt.subplots(figsize=(6, 4))
            sns.countplot(x='bomb_planted', data=df, ax=ax,
                palette=['#4C72B0', '#55A868'])
            ax.set_title("Распределение целевого признака",
                fontsize=12)
            ax.set_xlabel("bomb_planted", fontsize=10)
            ax.set_ylabel("Количество (count)", fontsize=10)
            ax.set_xticklabels(['0 (не заложена)', '1 (заложена)'])
            # Добавляем значения над столбцами
            for p in ax.patches:
                ax.annotate(f'{int(p.get_height())}',
                    (p.get_x() + p.get_width() / 2.,
                        p.get_height()),

```

```

        ha='center', va='bottom', fontsize=9)

st.pyplot(fig)

# Boxplot (time_left и ct_health)
with tab2:
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    # График 1: time_left vs bomb_planted
    sns.boxplot(x='bomb_planted', y='time_left', data=df,
ax=axes[0], palette='Set2')
    axes[0].set_title("Время раунда vs установка бомбы",
fontsize=10)
    axes[0].set_xlabel("bomb_planted", fontsize=9)
    axes[0].set_ylabel("Оставшееся время (сек)", fontsize=9)
    axes[0].set_xticklabels(['0', '1'])
    # График 2: ct_health vs bomb_planted
    sns.boxplot(x='bomb_planted', y='ct_health', data=df,
ax=axes[1], palette='Set2')
    axes[1].set_title("Здоровье СТ vs установка бомбы",
fontsize=10)
    axes[1].set_xlabel("bomb_planted", fontsize=9)
    axes[1].set_ylabel("Суммарное здоровье СТ (HP)",
fontsize=9)
    axes[1].set_xticklabels(['0', '1'])
    plt.tight_layout()
    st.pyplot(fig)

# Полная корреляционная матрица
with tab3:
    # Берём только числовые столбцы
    numeric_df = df.select_dtypes(include=[np.number])
    corr = numeric_df.corr()
    fig, ax = plt.subplots(figsize=(14, 10)) # увеличен размер
для читаемости
    sns.heatmap(corr, annot=True, fmt=".2f", cmap='coolwarm',
square=True, linewidths=0.5, ax=ax,
annot_kws={"size": 8})
    ax.set_title("Матрица корреляций (все признаки)",
fontsize=14)
    plt.xticks(rotation=45, ha='right', fontsize=8)

```

```

plt.yticks(rotation=0, fontsize=8)
plt.tight_layout()
st.pyplot(fig)

# PCA проекция
with tab4:
    st.write("### Проекция данных на 2 главные компоненты (PCA)")

    # Подготовка данных: исключаем целевую переменную и
нечисловые
    feature_cols = [c for c in df.columns if c != 'bomb_planted'
and df[c].dtype in [np.int64, np.float64]]
    X = df[feature_cols].copy()
    y = df['bomb_planted']

    # Масштабирование (StandardScaler)
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # PCA
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(X_scaled)

    # Создание DataFrame для plotly или matplotlib
    pca_df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
    pca_df['target'] = y.values

    fig, ax = plt.subplots(figsize=(8, 6))
    scatter = ax.scatter(pca_df['PC1'], pca_df['PC2'],
                        c=pca_df['target'], cmap='coolwarm',
                        alpha=0.6, edgecolors='k',
linewidth=0.5)
    ax.set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)')
    ax.set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)')
    ax.set_title('PCA проекция: bomb_planted = 0 (синий) / 1 (красный)')

    legend = ax.legend(*scatter.legend_elements(),
title="bomb_planted")

```

```

        ax.add_artist(legend)
        plt.tight_layout()
        st.pyplot(fig)
        st.caption(f"Объяснённая дисперсия: PC1 =
{pca.explained_variance_ratio_[0]*100:.1f}%, PC2 =
{pca.explained_variance_ratio_[1]*100:.1f}%")

# СТРАНИЦА 4: Прогнозирование (с поддержкой CSV)

elif page == "Прогнозирование":
    st.title("Получение прогноза модели машинного обучения")
    st.markdown("---")

    selected_model_name = st.selectbox("Выберите модель для инференса:",
list(models.keys()))
    model = models[selected_model_name]

    if model is None:
        st.error(f"Файл для модели '{selected_model_name}' не найден.")
    else:
        st.success(f"'{selected_model_name}' готов к работе.")

    # Переключатель способа ввода
    input_mode = st.radio("Способ ввода данных:", ["Ручной ввод
параметров", "Загрузить CSV-файл"])

    # РЕЖИМ 1: Ручной ввод

    if input_mode == "Ручной ввод параметров":
        st.markdown("### Конфигуратор параметров игрового раунда")
        col1, col2, col3 = st.columns(3)
        with col1:
            time_left = st.slider("Оставшееся время раунда (сек)",
0, 175, 90)
            map_val = st.selectbox("Карта (Идентификатор ID)", [0,
1, 2, 3, 4, 5, 6, 7])
            ct_score = st.number_input("Текущий счет команды СТ",
min_value=0, max_value=30, value=5)
            t_score = st.number_input("Текущий счет команды Т",

```

```

min_value=0, max_value=30, value=4)
    with col2:
        ct_health = st.slider("Суммарное здоровье СТ (HP)", 0,
500, 400)
        t_health = st.slider("Суммарное здоровье Т (HP)", 0,
500, 420)
        ct_armor = st.slider("Суммарные очки брони СТ", 0, 500,
300)
        t_armor = st.slider("Суммарные очки брони Т", 0, 500,
350)
        ct_players_alive = st.slider("Количество живых игроков
СТ", 0, 5, 4)
        t_players_alive = st.slider("Количество живых игроков
Т", 0, 5, 4)
    with col3:
        ct_money = st.number_input("Экономические ресурсы СТ
($)", min_value=0, max_value=100000, value=12000)
        t_money = st.number_input("Экономические ресурсы Т ($)",
min_value=0, max_value=100000, value=15500)
        ct_helmets = st.number_input("Количество шлемов у СТ",
min_value=0, max_value=5, value=3)
        t_helmets = st.number_input("Количество шлемов у Т",
min_value=0, max_value=5, value=4)
        ct_defuse_kits = st.number_input("Наборы сапера (Defuse
Kits) у СТ", min_value=0, max_value=5, value=2)

    ct_econom_power = ct_money // 1000 if ct_money > 0 else 0
    t_econom_power = t_money // 1000 if t_money > 0 else 0

    input_data = pd.DataFrame([
        'time_left':    time_left,      'ct_score':    ct_score,
't_score': t_score, 'map': map_val,
        'ct_health':    ct_health,      't_health':    t_health,
'ct_armor': ct_armor, 't_armor': t_armor,
        'ct_money': ct_money, 't_money': t_money, 'ct_helmets':
ct_helmets, 't_helmets': t_helmets,
        'ct_defuse_kits': ct_defuse_kits, 'ct_players_alive':
ct_players_alive, 't_players_alive': t_players_alive,
        'ct_econom_power': ct_econom_power, 't_econom_power':

```

```

t_econom_power
    })

    if df is not None:
        columns_order = [c for c in df.columns if c !=
'bomb_planted']
        input_data = input_data[columns_order]

        if st.button("Выполнить прогноз", type="primary"):
            pred = model.predict(input_data)[0]
            st.subheader("Результат классификации:")
            if pred == 1:
                st.error("Прогноз: бомба будет заложена")

            else:
st.success("Прогноз: бомба НЕ будет заложена")

        # РЕЖИМ 2: Загрузка CSV-файла

    else:
        st.markdown("### Загрузка данных из CSV")
        st.info("Файл должен содержать все признаки (те же колонки,
что и в обучающем датасете, без колонки bomb_planted).")

        uploaded_file = st.file_uploader("Выберите файл в формате
.csv", type=["csv"])

        if uploaded_file is not None:
            try:
                df_input = pd.read_csv(uploaded_file)
                st.write("Первые 5 строк загруженного файла:")
                st.dataframe(df_input.head())

                if df is None:
                    st.error("Не загружен эталонный датасет
'cs_clean.csv'. Невозможно проверить колонки.")
                else:
                    expected_cols = [c for c in df.columns if c !=
'bomb_planted']

```

```

missing_cols = [c for c in expected_cols if c
not in df_input.columns]

extra_cols = [c for c in df_input.columns if c
not in expected_cols and c != 'bomb_planted']

if missing_cols:
    st.error(f"В загруженном файле отсутствуют
необходимые колонки: {missing_cols}")
else:
    if extra_cols:
        st.warning(f"Найдены лишние колонки
(будут проигнорированы): {extra_cols}")

# Приводим порядок колонок и делаем
предсказание

X_pred = df_input[expected_cols].copy()

if X_pred.isnull().sum().sum() > 0:
    st.warning("Обнаружены пропущенные
значения. Они будут заполнены нулями (это может повлиять на качество).")
    X_pred = X_pred.fillna(0)

# Получение предсказаний и вероятностей
predictions = model.predict(X_pred)
if hasattr(model, "predict_proba"):
    probabilities =
model.predict_proba(X_pred)[:, 1]
else:
    probabilities =
np.full(len(predictions), np.nan)

# Формирование результата
result_df = X_pred.copy()
result_df["prediction"] = predictions
result_df["probability_%"] = (probabilities
* 100).round(1)

result_df["interpretation"] =
result_df["prediction"].apply(
    lambda x: "Бомба будет заложена" if x

```

```

== 1 else "Бомба НЕ будет заложена"
    )

    st.success("Прогноз выполнен успешно!")
    st.write("### Результаты предсказаний")
    st.dataframe(result_df,
use_container_width=True)

    csv_result =
result_df.to_csv(index=False).encode('utf-8')
    st.download_button(
        label="Скачать результаты (CSV)",
        data=csv_result,
        file_name="predictions.csv",
        mime="text/csv"
    )
except Exception as e:
    st.error(f"Ошибка при чтении файла: {str(e)}")

```